Documentation regarding the design of the Art Showcase and Request Website (ASRW)

Index:

1. Introduction

In this day and age, every artist needs a way to advertise and allow commissions of their own work, usually that need is fulfilled using a social media page and account, however this showcase method can sometimes be a little difficult to understand and compile all the information regarding the artist, such as their prior work with specific items, or the methods of contact, as such, an alternative to this is usually a website, from which the artist can both showcase their portfolio, allow clearer showcases of information, and more direct and organized communication with their clients.

The purpose of this document is to showcase the design and architecture of a possible implementation of the website proposed previously. The original website that this created was commissioned by the artist Helena Belas as a favor.

2. Requisites

Upon meeting with the client, and having a brief discussion with her, we concluded that the website should have these functional requirements:

a. The Website should be able to showcase all the works of the artist
b. The User should be able to see and search for the work showcased by the artist.
c. The User should be able to login into the website

d. The User should be able to view information regarding the type of requests by the Artist
e. The User should be capable of requesting work for the artist
f. The Artist should work as a single administrator
g. The Artist should be able to edit their own profile and what art is shown
h. The Artist should be able to see non-sensitive information regarding other users
i. The Artist should be able to see any information about the requests
j. The Artist should be able to add, edit and remove new types of works
k. The Artist should be able to put certain items at a discount for a period

Regarding non-functional requirements, there was no discussion regarding this topic, however, some information could be gleaned from more conversations and other possible problems.

a. Cost-effectiveness: The system should not cost more than 20€/month to maintain operational
b. Operability: The system should be able to be completely online and fully shutdown in less than 30 seconds
c. Availability: The system should be operational within expected time for around 99.9% of its running time
d. Traceability: The system should keep records regarding insertion of data by the Artist

With the requirements established, the minimum for a complete project should include the functional requirements, with the possibility of having the non-functional requirements operational as well.

3. Risk Analysis

With the requirements established, we can now begin to consider the risk that the Website may need to contend, and how these can be controlled

3.1. Possible Risks

Regarding possible failures and problems that may occur within the system, these were the ones that were identified, and how they were classified regarding probability and severity:

a. A Malicious Actor may gain access to a User account that should not belong to them (probability: moderate, severity: moderate)

b. A Malicious Actor may attempt some form of path traversal, or some different method, to access administrative pages (probability: moderate, severity: high)
c. The website is targeted by a DOS attack (probability: low, severity: moderate)
d. A misinput may cause the deletion of an item in the database (probability: high, severity: low)

3.2. Risk Control

With a set of possible risks into consideration, the next step would be to find ways to prevent, detect and mitigate these risks, and the resulting methods were:

Risk a.: One possible mitigation is to ensure that there is not any crucial information being stored and used, such as credit card information or an address, with a possible warning to the user regarding the repetition of passwords.

Risk b.: To prevent this issue, all pages and sections related to administration should have some form of authentication and authorization regarding the nature of the user, further prevention could be the need to confirm changes through an email, and only then could changes be accepted, as of detection, there could be the implementation of a logging system, that continuously write about any changes to the database, which helps to detect when the Artist was not responsible for some changes, and also allows some form of mitigation, as the inserted, edited or removed items can be read from the log, and simply reverted.

Risk c.: One method of detection could simply be utilization of a method that detects if the number of requests from a single IP is far beyond that of a normal amount, and to start blocking those requests if too many come out.

Risk d.: A simple prevention of this incident, could simply be the implementation of a dual confirmation system, that is, before the deletion of the request, ask the user if they are sure that they want to delete the item, as for mitigation, one that was referenced above for another risk, was the usage of a logging system, with the deletion of a random item being recorded, so that it can be restored soon after.

4. Use Cases

With a clear and concise idea of what is needed to be accomplished, another step to aid in the implementation, would be the creation of Use Cases, as to better design the way the system should and would be used. The resulting Use Cases were shown below:

Use Case 1: Visualization of the pieces of artwork

- Primary Actors: Users
- Scope: Artwork
- Stakeholders and Interests:
  - User: wishes to visualize some pieces of artwork
  - Artist: showcase the pieces of artwork to those who are interested
- Precondition: The User is already on the main page
- Success Guaranty: The User will be able to see all the items that match their description
- Main Success Scenario:
  - 1. The User goes to the search bar
  - 2. The User types the set of tags that they wish to find
  - 3. The User clicks the search button and sends the results to the Backend in the System
  - 4. The System queries the database for all artworks containing the selected tags
  - 5. The System compiles the artworks information with the images themselves
  - 6. The System sends the items to the Frontend
  - 7. The Frontend compiles the results to showcase the User
- Extensions:
  - 3a. The connection between the Backend and Frontend end unexpectedly before transfer
    - 3a1. The Frontend requests the User to wait a bit before repeating
  - 4a. There are no items related to the given tag
    - 4a1. The System returns an empty list of the items
    - 4a2. The Frontend shows the result page with a statement that they do not contain those items
  - 5a. The System could not match the information of one of the artworks to their image
    - 5a1. The artwork is assumed to not exist, and discarded
    - 5a2. The process resumes from step 6
  - 6a. The connection between the Backend and Frontend end unexpectedly before transfer
    - 6a1. The process is discarded and assumed as a failure

Use Case 2: Creation of a User account

- Primary Actors: Users
- Scope: User Management
- Stakeholders and Interests:
    - User: desires to create a personal account
- Precondition: The User is already on the main page
- Success Guaranty: The User will have a new account and become logged in
- Main Success Scenario:
    - 1. The User goes to the login page
    - 2. The User clicks on a button to create a new account
    - 3. The User fills the form with the required information
    - 4. The User clicks on a button to confirm the details and sends it to the Backend
    - 5. The System creates a new user account based on the given input
    - 6. The System logs in the User in the given account, and sends the confirmation to the Frontend
    - 7. The Frontend showcases the User the success of the operations and shows them logged in
- Extensions:
    - 4a. The connection between Frontend and Backend is severed abruptly
        - 4a1. The process is discarded, and the User is informed and asked to repeat the process after some time
    - 5a. The System already has an account with the given credentials
        - 5a1. The System returns a warning to the Frontend warning of the issue
        - 5a2. The User repeats from step 3
    - 6a. The connection between the Backend and Frontend end unexpectedly before transfer
        - 6a1. The process is repeated 3 more times, and if it still fails, warn the User of the failure

Use Case 3: User Login

- Primary Actors: Users
- Scope: User Management
- Stakeholders and Interests:
    - User: desires to login to a personal account
- Precondition: The User is already on the main page, and already has an account
- Success Guaranty: The User will have become logged in
- Main Success Scenario:
    - 1. The User goes to the login page
    - 2. The User inputs the username and password

- 3. The User clicks on a button to confirm and sends it to the Backend
    - 4. The System searches for the user account and compares if the password is correct
    - 5. The System logs in the User to the account, and sends the confirmation to the Frontend
    - 6. The Frontend showcases the User the success of the operations and shows them logged in
- Extensions:
    - 3a. The connection between Frontend and Backend is severed abruptly
        - 3a1. The process is discarded, and the User is informed and asked to repeat the process after some time
    - 4a. The System did not find an account with the given credentials
        - 4a1. The System returns a warning to the Frontend warning of the issue
        - 4a2. The User repeats from step 2
    - 4b. The password did not match that of the account
        - 4b1. The System returns a warning to the Frontend warning of the issue
        - 4b2. The User repeats from step 2
    - 5a. The connection between the Backend and Frontend end unexpectedly before transfer
        - 5a1. The process is repeated 3 more times, and if it still fails, warn the User of the failure

Use Case 4: Creation of a request

- Primary Actors: Users
- Scope: Art requests
- Stakeholders and Interests:
    - User: wants to make a new request for a piece
    - Artist: wishes to receive requests from possible clients
- Precondition: The User is already logged in
- Success Guaranty: A new request on the name of the User will be created
- Main Success Scenario:
    - 1. The User goes to the request page
    - 2. The User clicks on a button to create a new request
    - 3. The User fills the form with the required information and attachments
    - 4. The User clicks on a button to confirm the details and sends it to the Backend
    - 5. The System creates a new request based on the given input, and generates the extra information
    - 6. The System send the confirmation and full request generated to the Frontend
    - 7. The Frontend showcases the User the success of the operation

- Extensions:
    - 4a. The connection between Frontend and Backend is severed abruptly
        - 4a1. The process is discarded, and the User is informed and asked to repeat the process after some time
    - 6a. The connection between the Backend and Frontend end unexpectedly before transfer
        - 6a1. The process is repeated until the result is shown

Use Case 5: Artist Login

- Primary Actors: Artist
- Scope: User management
- Stakeholders and Interests:
    - Artist: wishes to log in as the artist themselves
- Precondition: The Artist is already on the main page, and already has an artist account
- Success Guaranty: The Artist will be logged into the artist's account
- Main Success Scenario:
    - 1. The Artist goes to the artist login page
    - 2. The Artist inputs the username and password
    - 3. The Artist clicks on a button to confirm and sends it to the Backend
    - 4. The System compares the artist account to see if they match
    - 5. The System logs in the Artist to the account, and sends the confirmation to the Frontend
    - 6. The Frontend showcases the Artist the success of the operations and shows them logged in
- Extensions:
    - 3a. The connection between Frontend and Backend is severed abruptly
        - 3a1. The process is discarded, and the Artist is informed and asked to repeat the process after some time
    - 4a. The given credential did not match that of the artist's account
        - 4a1. The System returns a warning to the Frontend depending on the issue
        - 4a2. The User repeats from step 2
    - 5a. The connection between the Backend and Frontend end unexpectedly before transfer
        - 5a1. The process is repeated 3 more times, and if it still fails, warn the User of the failure

Use Case 6: Artist inserts a new piece of art

- Primary Actors: Artist
- Scope: Artwork
- Stakeholders and Interests:
  - Artist: wishes to insert a new art piece
- Precondition: The Artist is already logged in
- Success Guaranty: The Artist will add a new piece of art to the Database
- Main Success Scenario:
  - 1. The Artist goes to the artist management page
  - 2. The Artist clicks the button to add artwork
  - 3. The Artist fills a form with the characteristics of the items, and attaches a picture of the artwork
  - 4. The Artist submits the form of the new artwork sending it to the Backend
  - 5. The System adds the new artwork to the Database and sends the confirmation to the Frontend
  - 6. The Frontend informs the Artist of the addition of the artwork
- Extensions:
  - 3a. Some of the fields of the process are incorrectly filled
    - 3a1. The Front End informs the Artist, and requests them to repeat the step
  - 4a. The connection between Frontend and Backend is severed abruptly
    - 4a1. The process is discarded, and the Artist is informed and asked to repeat the process after some
  - 5a. The connection between the Backend and Frontend end unexpectedly before transfer
    - 5a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure

Use Case 7: Artist updates a piece of art

- Primary Actors: Artist
- Scope: Artwork
- Stakeholders and Interests:
  - Artist: wishes to alter an art piece
- Precondition: The Artist is already logged in, there already exists the artwork that the Artist wishes to update
- Success Guaranty: The Artist will modify an existing piece of art on the Database
- Main Success Scenario:
  - 1. The Artist goes to the artist management page
  - 2. The Artist clicks the button to update artwork
  - 3. The Artist searches and selects the artwork to be updated

- 4. The System searches for that artwork
- 5. The System returns to the Frontend the details of the artwork
- 6. The Artist fills a form with the new characteristics of the item that are to be changed, and attaches a new picture of the artwork, if this is to be changed
- 7. The Artist submits the form of the new artwork sending it to the Backend
- 8. The System modifies the referred artwork in the Database and sends the confirmation to the Frontend
- 9. The Frontend informs the Artist of the update to the artwork

- Extensions:
  - 4a. The to be updated artwork is not found
    - 4a1. The Frontend warns the Artist that no items were found
  - 5a. The connection between the Backend and Frontend end unexpectedly before transfer
    - 5a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure
  - 6a. Some of the fields of the process are incorrectly filled
    - 6a1. The Front End informs the Artist, and requests them to repeat the step
  - 7a. The connection between Frontend and Backend is severed abruptly
    - 7a1. The process is discarded, and the Artist is informed and asked to repeat the process after some
  - 8a. The connection between the Backend and Frontend end unexpectedly before transfer
    - 8a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure

Use Case 8: Artist deletes or revoke a piece of art

- Primary Actors: Artist
- Scope: Artwork
- Stakeholders and Interests:
  - Artist: wishes to remove an art piece
- Precondition: The Artist is already logged in, there already exists the artwork that the Artist wishes to delete or revoke
- Success Guaranty: The Artist will delete or mark as ignorable an existing piece of art on the Database
- Main Success Scenario:
  - 1. The Artist goes to the artist management page
  - 2. The Artist clicks the button to delete/revoke artwork
  - 3. The Artist searches and selects the artwork to be removed

- 4. The System searches for that artwork
- 5. The System returns to the Frontend the details of the artwork
- 6. The Artist confirms if the artwork is the correct work, informs if the artwork is meant to be revoked or deleted, and sends back the response
- 7. The System either removes the artwork from the Database or stops that artwork from being shown, depending on the request that the artwork specified, and sends the confirmation to the Frontend
- 8. The Frontend informs the Artist of the successful deletions of the artwork

- Extensions:
  - 4a. The artwork meant to be deleted is not found
    - 4a1. The Frontend warns the Artist that no items were found
  - 5a. The connection between the Backend and Frontend end unexpectedly before transfer
    - 5a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure
  - 6a. The connection between Frontend and Backend is severed abruptly
    - 6a1. The process is discarded, and the Artist is informed and asked to repeat the process after some
  - 6b. The returned artwork is different from the one that should be deleted
    - 6b1. The Artist cancels the request, and starts over from step 3 with the proper parameters
  - 7a. The connection between Frontend and Backend is severed abruptly
    - 7a1. The process is discarded, and the Artist is informed and asked to repeat the process after some

Use Case 9: Artist browses the current requests

- Primary Actors: Artist
- Scope: Art Requests
- Stakeholders and Interests:
  - Artist: wishes to see the current requests
- Precondition: The Artist is already logged in
- Success Guaranty: The Artist will be able to see all current requests that meet the desired criteria of the Artist, if no criteria exist, then all requests will be shown
- Main Success Scenario:
  - 1. The Artist goes to the artist management page
  - 2. The Artist clicks on a button to search the current requests
  - 3. The Artist applies some filters from a list of those if desired and sends the request to the Backend

    - 4. The System creates a list based on all the filters and applies the desired filters on said list
    - 5. The System returns the final list to the Frontend
    - 6. The Frontend showcases the Artist all the found requests
- Extensions:
    - 3a. The connection between Frontend and Backend is severed abruptly
        - 3a1. The process is discarded, and the Artist is informed and asked to repeat the process after some time
    - 4a. There were no items that matched the requests and filters
        - 4a1. The System returns an empty list
        - 4a2. The Frontend reads the empty list and instead shows a page built to show that no items were found
    - 5a. The connection between the Backend and Frontend fails unexpectedly before transfer
        - 5a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure

Use Case 10: Artist browses the current Users

- Primary Actors: Artist
- Scope: User Management
- Stakeholders and Interests:
    - Artist: wishes to see the current users
- Precondition: The Artist is already logged in
- Success Guaranty: The Artist will be able to see all current Users that meet the desired criteria of the Artist, if no criteria exist, then all requests will be shown
- Main Success Scenario:
    - 1. The Artist goes to the artist management page
    - 2. The Artist clicks on a button to search the current Users
    - 3. The Artist applies some filters from a list of those if desired and sends the Users to the Backend
    - 4. The System creates a list based on all the filters and applies the desired filters on said list
    - 5. The System returns the final list to the Frontend
    - 6. The Frontend showcases the Artist all the found Users
- Extensions:
    - 3a. The connection between Frontend and Backend is severed abruptly
        - 3a1. The process is discarded, and the Artist is informed and asked to repeat the process after some time

- 4a. There were no items that matched the Users and filters
  - 4a1. The System returns an empty list
  - 4a2. The Frontend reads the empty list and instead shows a page built to show that no items were found
- 5a. The connection between the Backend and Frontend fails unexpectedly before transfer
  - 5a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure

Use Case 11: Artist creates a new type of artwork

- Primary Actors: Artist
- Scope: Artwork
- Stakeholders and Interests:
  - Artist: wishes to insert a new type of artwork
- Precondition: The Artist is already logged in
- Success Guaranty: The Artist will add a new type of artwork to the Database
- Main Success Scenario:
  - 1. The Artist goes to the artist management page
  - 2. The Artist clicks the button to add artwork type
  - 3. The Artist fills a form with the characteristics of the artwork type
  - 4. The Artist submits the form of the new artwork type, sending it to the Backend
  - 5. The System adds the new artwork type to the Database and sends the confirmation to the Frontend
  - 6. The Frontend informs the Artist of the addition of the artwork type
- Extensions:
  - 4a. The connection between Frontend and Backend is severed abruptly
    - 4a1. The process is discarded, and the Artist is informed and asked to repeat the process after some
  - 5a. The connection between the Backend and Frontend end unexpectedly before transfer
    - 5a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure

Use Case 12: Artist updates a new type of artwork

- Primary Actors: Artist
- Scope: Artwork

- Stakeholders and Interests:
    - Artist: wishes to alter a type of artwork
- Precondition: The Artist is already logged in, there already exists the artwork type that the Artist wishes to update
- Success Guaranty: The Artist will modify an existing artwork type on the Database
- Main Success Scenario:
    - 1. The Artist goes to the artist management page
    - 2. The Artist clicks the button to update artwork type
    - 3. The Artist searches and selects the artwork type to be updated
    - 4. The System searches for that artwork type
    - 5. The System returns to the Frontend the details of the artwork type
    - 6. The Artist fills a form with the new characteristics of the artwork type that are to be changed
    - 7. The Artist submits the form of the new artwork type sending it to the Backend
    - 8. The System modifies the referred artwork type in the Database and sends the confirmation to the Frontend
    - 9. The Frontend informs the Artist of the successful addition of the artwork type
- Extensions:
    - 4a. The to be updated artwork is not found
        - 4a1. The Frontend warns the Artist that no items were found
    - 5a. The connection between the Backend and Frontend end unexpectedly before transfer
        - 5a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure
    - 6a. Some of the fields of the process are incorrectly filled
        - 6a1. The Front End informs the Artist, and requests them to repeat the step
    - 7a. The connection between Frontend and Backend is severed abruptly
        - 7a1. The process is discarded, and the Artist is informed and asked to repeat the process after some
    - 8a. The connection between the Backend and Frontend end unexpectedly before transfer
        - 8a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure

Use Case 13: Artist deletes an artwork type

- Primary Actors: Artist
- Scope: Artwork
- Stakeholders and Interests:

    - Artist: wishes to remove an art type
- Precondition: The Artist is already logged in, there already exists the artwork type that the Artist wishes to remove
- Success Guaranty: The Artist will delete an existing artwork type on the Database
- Main Success Scenario:
    - 1. The Artist goes to the artist management page
    - 2. The Artist clicks the button to delete artwork types
    - 3. The Artist searches and selects the artwork type to be removed
    - 4. The System searches for that artwork type
    - 5. The System returns to the Frontend the details of the artwork type
    - 6. The Artist confirms if the artwork type is the correct one, and sends back the response
    - 7. The System removes the artwork the artwork type of the Database and sends the confirmation to the Frontend
    - 8. The Frontend informs the Artist of the addition of the artwork
- Extensions:
    - 4a. The artwork meant to be deleted is not found
        - 4a1. The Frontend warns the Artist that no items were found
    - 5a. The connection between the Backend and Frontend end unexpectedly before transfer
        - 5a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure
    - 6a. The connection between Frontend and Backend is severed abruptly
        - 6a1. The process is discarded, and the Artist is informed and asked to repeat the process after some
    - 6b. The returned artwork is different from the one that should be deleted
        - 6b1. The Artist cancels the request, and starts over from step 3 with the proper parameters
    - 7a. The connection between Frontend and Backend is severed abruptly
        - 7a1. The process is discarded, and the Artist is informed and asked to repeat the process after some

Use Case 14: Artist sets a discount on a type of artwork

- Primary Actors: Artist
- Scope: Art Requests
- Stakeholders and Interests:
    - Artist: wishes to apply a discount to the requests of an artwork type

- Precondition: The Artist is already logged in, there already exists the artwork type that the Artist wishes to put on discount
- Success Guaranty: The Artist will apply a discount on the requests for an existing artwork type on the Database
- Main Success Scenario:
    - 1. The Artist goes to the artist management page
    - 2. The Artist clicks the button to add discount
    - 3. The Artist searches and selects the artwork type to be discounted
    - 4. The System searches for that artwork type
    - 5. The System returns to the Frontend the details of the artwork type
    - 6. The Artist fills a form with the new characteristics of discount to be applied
    - 7. The Artist submits the form of the new discount, sending it to the Backend
    - 8. The System saves the new discount in the Database and sends the confirmation to the Frontend
    - 9. The Frontend informs the Artist of the successful addition of the discount
- Extensions:
    - 4a. The artwork to be discounted is not found
        - 4a1. The Frontend warns the Artist that no items were found
    - 5a. The connection between the Backend and Frontend end unexpectedly before transfer
        - 5a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure
    - 6a. Some of the fields of the process are incorrectly filled
        - 6a1. The Front End informs the Artist, and requests them to repeat the step
    - 7a. The connection between Frontend and Backend is severed abruptly
        - 7a1. The process is discarded, and the Artist is informed and asked to repeat the process after some
    - 8a. The connection between the Backend and Frontend end unexpectedly before transfer
        - 8a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure

Use Case 15: Artist marks one of the requests as finished

- Primary Actors: Artist
- Scope: Art Requests
- Stakeholders and Interests:
    - Artist: wishes to see the current requests

- Precondition: The Artist is already on the artist management page, with the list of current requests
- Success Guaranty: The request will be marked as finished
- Main Success Scenario:
    - 1. The Artist will look for the specific request
    - 2. The Artist will click on the finish request button, and send the request to the Backend
    - 3. The System will process the closing of the request
    - 4. The System will inform the result to the Artist
    - 5. The Frontend showcases the successful update, and updates the request list as well
- Extensions:
    - 2a. The connection between Frontend and Backend is severed abruptly
        - 2a1. The process is discarded, and the Artist is informed and asked to repeat the process after some time
    - 4a. The connection between the Backend and Frontend fails unexpectedly before transfer
        - 4a1. The process is repeated 3 more times, and if it still fails, warn the Artist of the failure

5. Architecture

With the Use Cases established, we can begin to apply the Use Cases established above, and use them to create a plan as architecture for the System that we are designing

5.1. General Architecture

Since this is a project with a relatively small scale, the general architecture should be relatively simple and small, to ensure that not only is the project relatively simple to understand but is also of relatively low maintenance costs. For this reason, simple Django architecture, separated into a few services, should be the best solution, these services would all be connected to a single database to store all relevant data.

The services in question would be the following:

- Artwork showcase: The main purpose of this service would be to manage the showcase of artworks, and other related pages
- Request manager: This service would be focused on managing the requests sent to the Artist, ranging from their creation to their conclusion

- User management: For managing all User and/or Artist specific requests, this service should be the best implementation

5.2. Database

With the general architecture broadly designed, the next step would be to create a database to better store the values and information regarding the system, and define their structure

The tables would be the following:

- User:
    - email: string (PK)
    - name: string
    - password: encrypted string
    - isArtist: boolean
- Artwork:
    - id: int (PK)
    - name: String
    - type: 1+ ArtworkType
    - tags: 1+ ArtworkTag
    - location: String
- ArtworkTag:
    - Id: int (PK)
    - name: String
    - sonOf: 0-1 ArtworkTag
- ArtworkType:
    - Id: int (PK)
    - name: String
    - price: float
    - averageTime: Int
- Discount:
    - Id: int (PK)
    - percentage: float
    - startDate: DateTime
    - endDate: DateTime
    - artworkType: ArtworkType
- ArtRequest:

    - Id: int (PK)
    - category: ArtworkType
    - deadline: DateTime
    - description: String
    - done: boolean
    - discount: float
- Annexes:
    - Id: int (PK)
    - package: File
    - artRequestReferenced: ArtRequest

5.3. URLs and API

With the database and architecture properly discussed and designed, we can now start planning how the API should be structured, and what each URL does, which leaves us with the following URLs:

- "asrw/artist_login"

This page's purpose is to allow the Artist to be able to login as an Artist

- "asrw/artist/add_discount"

This URL allows the logged in artist to be able to add discounts to the types of artwork desired

- "asrw/artist"

The Artist's main management page, where after logging in, the artist is able perform all their capabilities

- "asrw/artist/create_artwork"

Using this URL, the Artist can create a new Artwork

- "asrw/artist/create_artwork_type"

Allows a logged in Artist to create a new Artwork type

- "asrw/artist/delete_artwork/<artwork _id>"

Allows the Artist to completely remove an Artwork from the System

- "asrw/artist/delete_artwork_type/<artwork_type_id>"

Allows the Artist to completely remove an Artwork Type from the System

- “asrw/artist/finish_request/<request_id>”

Marks a Request as finished, making it stop appearing on non-finished searches

- “asrw/artist/revoke_artwork”

Marks an Artwork as revoked, stopping it from being searched for by Users other than the Artist

- “asrw/artist/search_artwork”

Allows the Artist to search for an Artwork based on specific tags or words

- “asrw/artist/search_artwork_type”

Allows the Artist to search for a specific Artwork Type based on tags and other words

- “asrw/artist/update_artwork/<artwork _id>”

Allows the Artist to update a specific item of a random Artwork

- “asrw/artist/update_artwork_type/<artwork_type _id>”

Allows the Artist to update a specific item of a random Artwork Type

- “asrw/create_acount”

Allows a User to create an account to create requests

- “asrw/create_request”

The page where a User may be able to create a new request

- “asrw/login”

Used by normal users to login into the Website, allowing the creation of Requests

- “asrw/main_page”

The home page for the Website, where the entry link should route one to

- “asrw/search_artwork”

The page where the Users may be able to search for a specific artwork

6. Conclusion

With the architecture concluded and a general idea of how the final product is supposed to run, we can now start the production of the Website, with further implementation decisions being noted in a separate document as to better separate the original intent from future decisions.

A few thing that were left out from this document that should be further explained would be, the CI/CD pipeline, the deployment strategy of the System and the type of Database System that would be used, as due to this moment, no clear decision as been made regarding these topics.